MachBoost: Conditional Acceleration for On-Device Text and Visual Models

Vistrit Pandey
vistrit@blinkfire.com

August 2026

Abstract

MachBoost is a local inference package for conditional acceleration. Its text paths draft from explicit context, reuse model state for an unchanged repository prefix, or use a target-verified block-diffusion drafter for selected fresh prompts. Visual requests can reuse state derived from unchanged image bytes or, experimentally, shorten a new image's visual sequence after Qwen3-VL completes early cross-modal fusion. A team gateway adds revision-aware private and shared memory, deterministic exact-request reuse, and budgeted external-provider fallback; these serving features are kept separate in the evaluation. Controlled repository-prefix experiments on an Apple M5 Pro report 2.97× and 3.23× median speedups across five changing questions on Qwen2.5 3B and 7B. A later audit uses an 8,754-file private production repository, a 7B model, and independent standalone threads. Reusing 75–77% of prompt tokens reduces median prefill by 3.75–3.84× and end-to-end time by 1.31–1.43× across six paired new-question runs. Decode time is unchanged, and all six pairs are byte-identical under greedy generation. On fresh 512-token outputs, Qwen3.5 4B BF16 block-diffusion decoding raises median throughput from 28.25 to 46.81 tokens/s across three prompt families, a 1.64× median; prompt medians range from 1.31× to 2.54×, and all three 128-token validation prefixes match native greedy output. A practical 9B 4-bit control reaches 1.32× with adaptive verification but does not always preserve the native MLX sequence. Separate context-drafting experiments on an Apple M1 Max are less consistent: a favorable 3B Qwen code fixture reaches 1.96×, while a broader Llama 3.2 3B audit records a 1.008× aggregate median and exact token equality in 20 of 21 pairs. Across six Qwen vision models, repeated-image paired medians range from 5.14× to 17.44×. A ten-image Qwen3-VL 8B TextVQA pilot with all caches disabled reaches a 1.67× aggregate speedup after retaining a median 35.12% of visual hidden states. Native and compressed paths each answer eight questions correctly, although normalized outputs agree on seven. The measurements support distinct, workload-dependent contracts rather than universal faster inference.

1 Introduction

Running LLMs on a laptop has become realistic, but it has not made autoregressive decoding disappear. A small model on Apple Silicon may fit in memory and still feel slow because the loop is serial: produce a token, update the prefix, run the model again. That bottleneck is especially visible in developer workflows, where the model often sits inside an editor, command runner, notebook, or local assistant.

Visual assistants add a different cost. One image may expand into roughly a thousand language-model input positions before the answer begins. Asking a second question about an unchanged screenshot, invoice, diagram, or camera frame can repeat both the vision encoder and the long

visual-token prefill. A first question cannot use that cache, but it may still carry redundant visual states through every language layer.

Speculative decoding offers a way around part of this loop. A cheap source guesses several future tokens; the target model checks the guess; accepted tokens are emitted without paying for one full target step each. Prior work usually gets the draft from another model, extra heads, or a reference sequence. MachBoost takes the reference idea and applies it to the local machine. If the user is asking about a repo, config, note, policy, or retrieved passage, the likely continuation may already be present in that local text.

A large repository introduces a separate prefill problem. Sending every file is both wasteful and likely to exceed the context window, while conventional retrieval still re-evaluates a long repository description on each question. MachBoost indexes bounded code chunks locally, constructs a deterministic file-and-symbol map, retrieves focused line windows, and arranges the prompt so the stable map precedes the changing evidence. A resident MLX model can then restore the longest exact prompt prefix before evaluating the new suffix.

MachBoost does not try to accelerate every prompt or modality with one trick. It accelerates grounded text when a verifier can accept useful drafts, repository prefill when a stable code map can be reused, repeated-image queries when the visual input has not changed, and selected Qwen3-VL first-view requests through an opt-in lossy path. Ordinary requests still follow the backend's native implementation.

This paper contributes the following:

- a context-only drafter and initial-candidate gate that avoid treating the prompt wrapper itself as external evidence;
- a cache-aware MLX verifier that fuses continuation and draft scoring, rewinds rejected cache entries in place, and resumes native decoding when useful context ends;
- a paired benchmark against native `mlx-lm` generation with raw rows, run-order alternation, environment provenance, and exact token comparisons;
- a resident local server with warm model lifecycle, streaming Ollama-compatible and OpenAI-compatible endpoints, and raw completion support for editor integrations;
- a Git-aware local repository index that combines a deterministic file-and-symbol map with bounded, line-addressable retrieval;
- a repository-prefix path that restores native MLX prompt state only for an exact token prefix, plus paired 3B and 7B artifacts over different questions;
- a revision- and dependency-aware team memory ledger with private and administrator-published scopes, deterministic exact-request reuse, and machine-readable avoided-work counters;
- an architecture-aware MLX-VLM adapter with content-addressed image identity, projected-feature reuse where the model interface preserves all required tensors, and image-scoped prompt state;
- a whole-prefix checkpoint path for Qwen3.5's mixed recurrent/attention cache, plus a conservative prompt-state-only path for Qwen3-VL's deep-stack visual features;
- paired repeated-image artifacts for six Qwen models that record cache hits, matched prefix length, first-token latency, raw outputs, and expected-answer checks;

- an experimental Qwen3-VL wrapper that performs query-weighted spatial merging only after all early deep-stack visual injections, with per-request retention telemetry;
- shape-, prompt-, and depth-aware visual policies, including deterministic random controls and an offline selector that can disable unsupported workloads;
- paired bootstrap intervals and atomic checkpoints for multi-dataset first-view ablations;
- an FFmpeg-based temporal adapter that selects chronological scene-change frames before the existing multi-image VLM path;
- a paired unique-image TextVQA pilot against the same model's full-token path, with both cache systems disabled and raw outputs retained;
- a resident MLX integration of DFlash block-diffusion drafting, adaptive verification, paired same-target benchmarks, and a strict greedy-output validation gate;
- negative experiments with a small autoregressive draft model, an MTP draft path, an Apple Neural Engine draft, and lossless weight compression.

2 Background and Related Work

Speculative decoding was introduced as a way to accelerate autoregressive transformer inference without changing outputs by drafting candidate tokens and verifying them in parallel with the target model [12]. Speculative sampling independently developed a related draft-and-verify method for sampled decoding and reported 2–2.5× speedups in a distributed setting [5]. Subsequent work explored architecture-level or self-drafting variants. Medusa adds multiple decoding heads to predict future tokens and verifies candidate trees [4].

MachBoost is closest to methods that exploit references or prompt overlap rather than an auxiliary draft model. LLMA observes that outputs in retrieval and reference-grounded scenarios often share spans with the input reference, then verifies copied spans to preserve greedy output [26]. Prompt Lookup Decoding replaces the draft model with n-gram matching over the prompt [19]; PLD+ extends that idea by using language-model artifacts such as attention or hidden states to rank candidate spans [20]. Hugging Face Transformers exposes prompt lookup through `prompt_lookup_num_tokens` in assisted generation [10]. MachBoost differs mainly in packaging and operating assumptions: it treats local files and retrieved snippets as ordinary inputs to the accelerator, keeps benchmark data machine-readable, and makes low-overlap prompts a measured fallback case rather than an error.

Results above 2× elsewhere generally require more than process tuning. ReDrafter trains a recurrent draft conditioned on target hidden state and reports up to 2.3× on Apple Silicon [27]. QuantSpec changes the draft's weight and KV-cache representation and reports speedups up to about 2.5× in long-context inference [21]. Mirror-SD maps complementary draft and target work across GPU and NPU devices and reports 2.8–5.8× on server-scale models [3]. At the runtime layer, BaseRT's native Metal kernels report up to 1.35× over MLX decode rather than a universal 2× [23]. These are different operating points: trained model changes, heterogeneous execution, or low-level kernels. MachBoost's current context path needs none of them, but only applies when the requested continuation overlaps local text.

DFlash replaces serial autoregressive drafting with a block-diffusion model and reports up to 7.1× speedup when a long proposed block is accepted [6]. Its speed still depends on target verification cost, draft cost, and acceptance. A recent consumer-hardware study reaches at most 1.61× and

finds that three of five speculative configurations regress [7]. AngelSpec addresses workload heterogeneity by training separate conversational MTP and code/mathematics block-diffusion drafters, then allocating verification depth with a profiled cost model [14]. MachBoost does not train a new drafter. It integrates a published DFlash pair into a local server and measures where adaptive block selection survives Apple Silicon's runtime costs.

The same distinction appears outside text generation. Diffusion workloads can reuse intermediate transformations across denoising steps; EasyCache reports 2.1–3.3× for several video models [29]. That mechanism does not transfer directly to autoregressive text decoding. It does, however, support a broader product design in which one package selects a backend-specific accelerator rather than pretending that a single algorithm fits text, image, audio, and video models.

Attention-state reuse is established. Prompt Cache stores reusable attention states for prompt modules and reports large first-token latency reductions on repeated prefixes [8]. MLX-VLM supplies prompt-state reuse, automatic prefix caching, and projected-feature caching [15]. The repository workspace studied here is an application of prefix caching, not a new cache primitive: its contribution is a local index and prompt layout that preserve a large exact prefix while query-specific code evidence changes. MachBoost composes existing cache facilities behind content identity and model-specific safety rules. The engineering question is whether one local serving layer can apply them without silently dropping architecture-specific visual or recurrent state.

Visual token reduction is also an active field rather than a MachBoost invention. AdaptInfer uses dynamic text guidance and layer-dependent pruning [28]. Wang et al. report that visual-token usefulness changes with depth and task, describing an architecture- and workload-dependent "information horizon" [22]. ZOO-Prune estimates token sensitivity without training and reports up to 2.30× end-to-end acceleration while pruning as much as 94.4% of visual tokens [11]. The first-view mechanism studied here is simpler: it uses no calibration data, gradients, or model changes, and targets one MLX Qwen3-VL implementation. Its contribution is the post-deepstack integration and local paired evidence, not the general idea of dropping or merging visual tokens.

Video introduces temporal redundancy before visual tokens reach the language model. PruneVid merges spatiotemporal tokens and uses question relevance to remove more than 80% of visual tokens in its reported experiments [9]. EchoPrune combines query relevance with temporal reconstruction error and reports a 5.6× prefill speedup for Qwen2.5-VL-7B while increasing the number of observed frames under a fixed token budget [13]. MachBoost's current video adapter is intentionally weaker: it uses RGB frame differences before model encoding and has no learned or hidden-state correspondence. It is a product-facing ingestion baseline, not a competing video-token result.

The implementation targets common local inference stacks. Hugging Face Transformers provides the broad model API surface for Python inference [24]. MLX is an Apple Silicon-oriented array framework and runtime used here for Mac-native evaluation [1]. Text measurements use a Qwen-family model converted for MLX; Qwen3 is the relevant public model-family report [25]. Visual measurements cover Qwen2.5-VL [2], Qwen3-VL [16], and Qwen3.5 [17].

3 Problem Setting

Let M be a target autoregressive model and let x be the prompt token sequence. Greedy decoding emits

$$y_t = \arg\max_v M(v \mid x, y_{<t})$$

for each generation step t. MachBoost receives a corpus C derived from prompt text, retrieved context, local files, or user-provided strings. The goal is to reduce wall-clock latency while returning the same sequence y that greedy decoding with M would return.

This paper stays with greedy decoding. Sampling-compatible speculative decoding is possible in principle, but greedy argmax verification gives a simple correctness check: the accelerated sequence must match the serial baseline token-for-token.

For a repository revision R, let $m(R)$ be a deterministic file-and-symbol map and let $e(R, q)$ be evidence retrieved for question q. The complete prompt is arranged as

$$x(R, q) = s \| m(R) \| e(R, q) \| q,$$

where s is fixed instruction text. For two questions over the same revision, the model state through $s \| m(R)$ can be reused when those tokens match exactly. MachBoost evaluates the remaining suffix and then runs the same native greedy decoder. This contract concerns prefill latency; it makes no claim about faster generation after the first token.

For repeated visual queries, let I be an image and q_i a sequence of questions sent to one resident vision-language model V. With identical image bytes, the desired invariant is

$$\text{CachedVLM}(V, I, q_i) = \text{NativeVLM}(V, I, q_i)$$

under the same deterministic generation settings. In practice, cold full-prefill and warm short-suffix kernels can follow different floating-point reduction paths, so we record both literal equality and task-answer agreement. No reuse is permitted across distinct content identities.

The first-view path has a different contract. Let P_r compress visual hidden states to an approximate retention ratio r after early fusion. We measure

$$\text{CompressedVLM}(V, I, q, r) = V_{>k}(P_r(V_{\leq k}(I, q)))$$

against the unmodified full-token path for latency and task score. Equality is measured but not guaranteed: merging changes the language model's hidden sequence. This distinction prevents the exact cache result and the approximate first-view result from being combined into one quality claim.

4 Method

The implementation has a drafter, a verifier, and an escape hatch to native generation. The escape hatch is important: speculative decoding is a conditional optimization, not a replacement decoder.

4.1 Drafting from Nearby Text

The drafter indexes token n-grams from a corpus C. At generation time it looks at the current prefix $p = x \| \hat{y}$, searches for suffixes of p that occur in C, and proposes the tokens that followed the best matching span. Longer suffix matches are preferred, with draft length as a secondary signal. Only explicit context is indexed in the reported adaptive mode. An earlier implementation indexed the concatenation of prompt and context; because benchmark prompts already displayed the source passage, that design could draft from the prompt wrapper and blur the boundary between available context and generated history.

Before entering the verifier loop, MachBoost asks whether the generation boundary has a context candidate. If not, the default profile calls native `mlx-lm` generation immediately. An opt-in re-entry profile generates a bounded number of target tokens, checks the context index again, and either enters block verification or resumes native generation from the live cache. Re-entry can recover copied answers whose first token begins a span in retrieved text. It also changes the cache trajectory, so version 0.2.0 disables it by default.

4.2 Verification Against the Target Model

Given a prefix p and draft $d = (d_1, \ldots, d_k)$, the verifier checks how many draft tokens equal the target model's greedy continuation. A verifier-capable runtime scores a block in one target call and compares target argmax tokens at the relevant positions. If the first a draft tokens are accepted, they are committed. On a mismatch, the target's residual token is emitted. When the whole block is accepted, the verifier also returns the next target token already present in the logits. The following round scores that pending token together with the next draft, avoiding a separate target call between accepted blocks.

Under deterministic target arithmetic, the intended invariant is:

$$\text{Boost}(M, x, C, T) = \text{Greedy}(M, x, T)$$

for a maximum generation length T. Batched and scalar floating-point paths can still choose different argmax tokens at a near tie, so this is tested rather than assumed. Every benchmark row records an `output_match` flag from the native and accelerated token IDs.

4.3 Block-Diffusion Drafting for Fresh Prompts

The context drafter cannot help when the continuation is absent from local text. For selected Qwen targets, MachBoost therefore exposes a separate DFlash backend. A lightweight diffusion model proposes up to 16 future tokens in parallel. The target scores the proposal and commits only the accepted prefix plus the target residual at the first rejection. No token chosen solely by the draft is returned.

The default verifier adapts its proposed block length from recent acceptance and measured cycle cost. A fixed 16-token verifier has higher upside when acceptance stays high, but it can spend more time verifying rejected positions than serial decoding would have spent generating the accepted prefix. The resident server therefore treats the accelerated pair as its own model alias rather than silently replacing a native alias. Both target and draft weights must be present before the catalog marks the alias ready.

Target approval is not by itself evidence that two runtime paths produce the same token sequence. Batched verification and scalar native decoding can reach a near-tied argmax through different floating-point reductions. MachBoost reruns a bounded greedy continuation through both paths, records token hashes and the first different position, and makes the benchmark exit nonzero on divergence. This gate tests runtime equivalence; it does not estimate semantic answer quality.

4.4 Keeping the Cache Path Stable

Verification writes candidate entries to the MLX prompt cache. Accepted entries remain useful; rejected entries must not survive. MachBoost trims the rejected suffix in place when the cache type permits it. If safe trimming is unavailable, the service reconstructs the accepted prefix. Prompt initialization follows `mlx-lm`'s split prefill trajectory, processing the final prompt token separately. Once no further context candidate exists, the service hands the live verifier cache to `mlx_lm.generate_step`; the remaining response therefore uses the runtime's optimized asynchronous decoder instead of a Python-level `argmax`/synchronize loop.

4.5 Indexing and Reusing Repository Prefixes

Repository discovery uses `git ls-files` when possible, preserving normal ignore rules. Symlinks, likely credential files, binaries, and oversized files are rejected before indexing. Remaining files are split into overlapping, line-addressable chunks and stored in SQLite FTS5 with extracted symbol names. Reindexing compares file metadata and content digests, then updates only changed or deleted entries.

A workspace revision combines the Git commit, when available, with a digest of the indexed path-and-content manifest. The content component matters for dirty working trees and untracked files: changing indexed source invalidates exact responses and prompt-state namespaces even when `HEAD` has not moved.

The context begins with a deterministic capsule containing repository metadata, sorted paths, and extracted symbols. Query terms select FTS rows, but MachBoost sends focused windows around matching lines instead of every complete chunk. Dynamic evidence is capped at 8,000 characters and follows the capsule. This ordering trades some dynamic context capacity for a longer reusable prefix; file and line ranges remain attached to every selected window.

The MLX service keeps a bounded LRU of native prompt states. For a new prompt, it finds the stored entry with the longest exact token prefix, trims a copied state to that boundary, and evaluates only the suffix. Cache entries are keyed to the loaded service and bounded by both count and estimated bytes. Ordinary chat leaves this cache disabled; repository requests opt in. Cache types that cannot be safely trimmed, including the tested Qwen3.5 hybrid recurrent state, fall back to native prefill.

For team requests, prompt-state keys use the organization and content-addressed workspace revision rather than a conversation identifier. Independent threads therefore share only the stable repository/request prefix. Their messages, retrieved evidence, tool inputs, and generated text remain in the changing suffix and are evaluated normally.

4.6 Namespaced Team Memory and Exact Reuse

The team gateway does not concatenate employee transcripts into one shared context. A memory namespace includes organization, workspace, scope, repository revision, and, for private records, principal identity. Completed workspace requests can write a bounded redacted summary with citations and dependency digests. Retrieval searches private and shared namespaces independently under a fixed character budget, then rejects records whose revision or dependency digests no longer match the current index. Only an administrator may publish a shared record.

Exact-response reuse is a separate opt-in path. Its key includes namespace, workspace revision, model, and normalized request. It is restricted to non-streaming, temperature-zero requests without tools or images. A hit returns the recorded response without model execution and accounts for avoided prompt tokens, completion tokens, and configured cost. This is semantic reuse only in the literal repeated-request case; related questions use retrieved memory but still invoke the model.

4.7 Architecture-Aware Reuse for an Unchanged Image

The visual adapter computes a SHA-256 identity from the image contents. Local file metadata memoizes hashing but does not define identity; base64 payloads, data URLs, and in-memory images are hashed directly. The first bounded LRU maps this identity to the projected output of the vision tower,

$$h(I) \longmapsto E_V(I).$$

On a miss, the vision tower processes the pixel tensor and image grid. On a hit, the stored features enter through `cached_image_features`. Qwen2.5-VL and Qwen3.5 can use this path. Qwen3-VL also emits deep-stack visual tensors; retaining only its primary projected tensor is incomplete, so MachBoost disables feature-only reuse for that family.

Feature reuse still leaves hundreds of visual positions for the language model to prefill. A second bounded map associates $h(I)$ with reusable prompt state. Pure attention models can retain the matching KV prefix. Qwen3.5 is different: its language model interleaves ordinary KV layers with recurrent linear-attention arrays. Rewinding only keys and values leaves the recurrent arrays at the wrong point. MachBoost uses MLX-VLM's exact checkpoint facility for that family, storing a complete cache snapshot shortly before the question suffix and restoring both cache types together. A preliminary KV-only implementation changed answers and was discarded.

All state belongs to one loaded model instance and is cleared on reset or unload. This avoids cross-model tensor reuse and makes image replacement an ordinary miss. The method does not alter parameters or generated-token decoding.

4.8 Post-Fusion Compression for a New Image

Qwen3-VL supplies a primary visual sequence and additional deep-stack visual embeddings to its language model. Removing tokens before those injections changes their alignment, so MachBoost first executes every configured deep-stack injection. The measured profile compresses after layer 3 and before the remaining 33 language layers. Version 0.5 can delay compression to a later requested boundary, but never moves it before the final deep-stack injection.

For each contiguous image segment, visual positions are grouped by temporal index and a spatial block width $g = \lceil \sqrt{1/r} \rceil$ derived from retention ratio r. The mean hidden state of up to 48 trailing text positions forms a query vector. Within each spatial group, cosine similarity to this query is converted to a softmax weight and the weighted visual states are merged into one representative. Adaptive mode ranks groups by internal directional dispersion and leaves the most diverse groups unmerged until the requested visual-token budget is met. A configurable bucket rounds the target count upward to reduce shape churn. Text positions are never removed.

The wrapper updates position identifiers and attention masks to the retained sequence. It records original and retained sequence lengths, visual-token counts, layer index, and actual retention ratio. Requests using this path bypass prompt and visual caches so the measured behavior cannot depend on an earlier image. The implementation delegates to the original MLX-VLM model unchanged when the option is off, the request is not batch one, or no visual positions are present.

4.9 Automatic Policy and Offline Calibration

The request policy uses prompt terms and image signals to classify general, document/text, chart, spatial, and multi-image workloads. Image signals include maximum edge, grayscale entropy, and edge density; they are computed only when automatic selection is requested. The selected record contains mode, retention ratio, layer boundary, token bucket, source, and reason. Manual `merge` and `adaptive` modes remain available, while deterministic `random` selection serves as a token-count control.

An ablation artifact shares one native row across all candidates for an image, rotates execution order, and reports paired bootstrap intervals for aggregate wall-time speedup, median paired speedup, and task-score difference. The offline calibrator groups rows by the policy's observed workload class. A candidate must clear minimum pair count, the lower 95% speedup bound, maximum task-score loss, and minimum normalized output agreement. Random controls are excluded

from deployment. If no candidate passes, the generated workload profile has `mode=off`.

4.10 Temporal Frame Selection

For a video Z, FFmpeg samples an ordered frame sequence $F = (f_1, \ldots, f_n)$ at a requested rate. Each frame is reduced to a 64 by 64 RGB thumbnail. MachBoost computes mean absolute RGB difference between consecutive thumbnails, keeps the first and last frames, and retains intermediate frames above a threshold. If the set exceeds the frame budget, frames with the largest change scores are kept and then restored to chronological order. A uniform sampler over the same extracted sequence supplies the benchmark control.

Extraction is cached under a key derived from video path, size, modification time, and sample rate. The selected files are ordinary ordered image inputs to MLX-VLM. This lowers frame count before vision encoding but does not model optical flow, query relevance, or short events. The adapter reports sampled and selected counts, timestamps, change scores, cache state, and selection time.

4.11 Deciding When to Use the Drafter

Drafting from local text is sometimes the wrong choice. Open-ended prompts often accept only a few draft tokens, so verification overhead dominates. MachBoost therefore exposes a benchmark step. For a prompt family, it measures:

- token match between baseline and boosted output,
- wall-clock speedup,
- accepted draft tokens divided by generated tokens,
- target-call or forward-call reduction.

The high-level API can also calibrate an accelerator across representative prompts:

```
from machboost import Accelerator, GatePolicy

boost = Accelerator.from_mlx(model, context_paths=["./docs", "./src"])
calibration = boost.calibrate(
    prompts,
    max_tokens=32,
    gate_policy=GatePolicy(min_speedup=1.05,
                           min_acceptance_rate=0.10),
)
text, stats = boost.generate(prompt, max_tokens=128)
```

If the measured workload fails the policy, `generate` uses the native backend path. Calibration and the per-request initial-candidate gate serve different purposes: calibration enables a workload family, while the gate prevents verifier overhead on an individual request with no draft at its boundary.

5 Implementation

MachBoost is implemented as a small Python package with no required runtime dependencies. Optional extras install backend-specific dependencies:

- `machboost[mlx]` for MLX and `mlx-lm`,
- `machboost[hf]` for PyTorch and Transformers,
- `machboost[vision]` for MLX-VLM,
- `machboost[dflash]` for target-verified block-diffusion decoding,
- `machboost[video]` for Pillow-based frame comparison, with FFmpeg supplied by the host,
- Ollama HTTP support for benchmarking and capability detection.

The core package defines a `StepService` protocol with `next_token(prefix)` and an optional `verify(prefix, candidate)` hook. A service with only `next_token` can use the wrapper but cannot skip target work. A verifier can accept several draft tokens per target call. The high-level `Accelerator` loads MLX or Hugging Face models, reads strings, files, or directories as context, and exposes `benchmark`, `calibrate`, and `generate`.

Version 0.2.0 added a resident HTTP process; version 0.5.1 gives idle models a five-minute lifetime by default and retains them indefinitely only when requested. The process owns loaded accelerators, serializes requests per model, supports explicit unloading, and performs a short text-model compile warm-up before the first prompt. The CLI can start the server automatically, preload an alias such as `qwen2.5:3b`, and issue chat or raw completion requests without reloading weights. The server exposes `/api/chat`, `/api/generate`, model lifecycle endpoints, and OpenAI-compatible chat and completion endpoints. Native MLX streaming forwards `mlx-lm`'s already-detokenized text segments; verified multi-token spans use a stateful streaming detokenizer. This avoids the earlier cost of decoding the entire token prefix once per emitted token.

Accelerated aliases such as `qwen3.5:4b-dflash` use the same standard model field as other endpoints. Pulling one downloads its BF16 target and paired draft; the server keeps both resident behind the existing bounded queue. This is an explicit model selection. MachBoost does not silently substitute it for the smaller 4-bit `qwen3.5:4b` alias.

The current server also owns the repository index and exposes workspace registration, reindex, search, listing, and deletion routes. Chat and generation requests can provide a workspace identifier through either the Ollama-compatible payload or an OpenAI-compatible extension. Retrieval context is inserted into the actual model prompt, response metadata carries the selected file and line citations, and an affinity key prefers the same model replica for later workspace requests. The SwiftUI macOS client persists a workspace identifier per conversation and provides repository selection, refresh, and removal controls.

The same local SQLite database stores team-key hashes, memory metadata, exact responses, provider configuration, and usage counters. Plaintext employee keys are returned once, while external-provider keys remain in process memory or the macOS Keychain. Provider routing supports local-only, local-first, external-first, and external-only policies. Fallback is permitted only for transient overload, timeout, connection, or selected upstream errors; budget, authentication, and request-validation errors fail closed. The native app exposes memory metrics and provider administration without persisting secrets in the database.

The 0.3 series adds vision-model aliases, image arguments in the CLI and Python client, Ollama-style image payloads, and OpenAI-style image content parts. The current alias set includes

Qwen2.5-VL 3B, Qwen3-VL 2B/4B/8B, and Qwen3.5 0.8B/4B/9B. MLX objects are created and used on one dedicated worker thread because MLX arrays and streams are thread-affine. The worker owns model load, image preparation, cache lookup, generation, and cache release. Version 0.4 adds the opt-in `vision_tokens` mode and retention telemetry. Version 0.5 adds automatic and calibrated policies, random and shape/depth ablations, and temporal video-frame inputs. Post-fusion compression remains restricted to Qwen3-VL because the wrapper depends on that family's layer and deep-stack interfaces.

The Ollama HTTP adapter remains a wrapper: Ollama's public generation API does not expose the target token IDs, logits, mutable KV cache, and block-verification hook this implementation needs. Native integration would require a runner-level change rather than an HTTP option.

6 Evaluation

6.1 Machine and Model

Measurements were collected on an Apple M1 Max with 10 CPU cores and 32 GB unified memory, running macOS 26.5.2. The environment used Python 3.13.3, MLX 0.31.2, `mlx-lm` 0.31.3, and MLX-VLM 0.5.0. Text experiments used `mlx-community/Qwen2.5-3B-Instruct-4bit` and `mlx-community/Llama-3.2-3B-Instruct-4bit`. The repeated-image matrix used MLX 4-bit conversions of Qwen3-VL 2B/4B/8B and Qwen3.5 0.8B/4B/9B. The first-view pilot used `mlx-community/Qwen3-VL-8B-Instruct-4bit`. Only one model was resident during each run. Model fetch and load time were recorded but excluded from paired request latency.

Repository-prefix measurements use a separate Apple M5 Pro with 18 CPU cores and 48 GB unified memory, running macOS 26.6. The environment uses Python 3.13.3, MLX 0.31.2, and `mlx-lm` 0.31.3. The tested models are `mlx-community/Qwen2.5-3B-Instruct-4bit` and `mlx-community/Qwen2.5-7B-Instruct-4bit`. The JSON artifacts store this machine and package provenance.

Fresh-decode measurements use the same M5 Pro and software versions plus `dflash-mlx` 0.1.10. The principal target is `mlx-community/Qwen3.5-4B-MLX-bf16`; a Qwen3.5 9B 4-bit conversion supplies a practical quantized control. The draft weights are the corresponding `z-lab` DFlash repositories. Model load is excluded. Throughput tests force 512 generated tokens to expose steady-state decode; a separate 128-token greedy gate compares native and accelerated token sequences.

The benchmark requests 64 greedy tokens for three fixture families: literal Python continuation, retrieval-grounded answering, and open-ended writing. Each family is repeated five times with a fresh nonce under both default and re-entry profiles. Baseline and accelerated order alternate. Wall time includes prompt processing. The baseline calls `mlx-lm`'s native streaming generator; it is not MachBoost's `next_token` loop. The artifacts `results/mlx_native_default_qwen25_3b_20260713.json` and `results/mlx_native_reentry_qwen25_3b_20260713.json` contain prompts, token previews, raw times, package versions, hardware data, and each run's order.

6.2 Repository Workspace Prefix Results

The workspace harness indexes one 183-file revision into 961 chunks, primes the accelerated service once, and then measures six questions. The first measured row exactly repeats the prime; the other five questions are different from the prime and from one another. Both services wrap the same loaded model and receive the same complete prompt. The baseline disables native prefix reuse, while the accelerated service restores the longest exact prompt state. Request order alternates, generation is greedy, and each response is capped at 16 tokens.

Model	Exact	All rows	Different questions	Native	MachBoost
Qwen2.5 3B	6/6	3.021×	2.971×	3.144 s	1.024 s
Qwen2.5 7B	6/6	3.282×	3.232×	6.587 s	1.998 s

Table 1: Repository-prefix results. "All rows" is the median of six paired ratios and includes one exact repeat of the priming request. The different-question median covers the other five rows. Native and MachBoost columns are all-row median wall times.

The median complete prompt contains 10,405 tokens, of which the accelerated path reuses 7,901. Different-question speedups range from 2.873× to 3.078× on 3B and 3.090× to 3.365× on 7B. The repeated prime reaches 13.055× and 16.637×, respectively. Every pair produces the same generated token IDs. The artifacts `results/workspace_prefix_qwen25_3b_20260729.json` and `results/workspace_prefix_qwen25_7b_20260729.json` retain prompt sizes, cache counters, timings, output hashes, citations, execution order, model, machine, packages, and repository revision.

These rows test exact prefill reuse, not answer quality or faster decoding. The questions retrieve different evidence but share a long repository map. A first workspace request, a request outside that workspace, or a long answer dominated by decode does not inherit the reported ratio.

6.3 Private-Repository Cross-Thread Audit

To check whether the controlled repository result survives a more realistic shape, a second harness indexes an 8,754-file private production monorepo into 26,123 chunks (87.1 MB of indexed text). Cold indexing takes 156.28 seconds; a later incremental pass scans 9,021 candidates and updates one changed file in 6.75 seconds. Paths, prompts, retrieved code, and generated answers are omitted from the public artifact.

The model is Qwen2.5 7B Instruct 4-bit on the same 48 GB M5 Pro. Each target is a new standalone thread. Exact-response reuse and semantic memory are disabled. Within a pair, native and MachBoost paths use the same resident model, retrieved evidence, complete prompt, output limit, and greedy decoding; only shared native prompt-state reuse differs. Order alternates over three measured pairs per scenario.

Scenario	Prompt	Reused	Prefill	Wall	Exact	Rubric
Adjacent code	4,243	3,269 (77.1%)	3.835×	1.314×	3/3	4/8–4/8
Other subsystem	4,327	3,257 (75.3%)	3.754×	1.426×	3/3	2/7–2/7

Table 2: Independent-thread private-repository results. "Exact" is byte equality between paired greedy outputs. Rubric columns show baseline and MachBoost hits on a small implementation-concept checklist, not a general code quality score.

For the adjacent-code question, median prefill falls from 2.306 to 0.605 seconds, time to first token from 2.430 to 0.726 seconds, and wall time from 7.034 to 5.377 seconds. Median decode is 4.472 seconds natively and 4.469 seconds with reuse. The other-subsystem question cuts prefill from 2.435 to 0.648 seconds and wall time from 5.995 to 4.217 seconds; decode remains about 3.33 seconds. This control is useful because the two threads concern different parts of the repository. Topic overlap between employee chats is not required; the reused object is the deterministic workspace prefix. A first request for a new revision still pays full prefill.

The same run separates two easy-to-confuse features. Three exact deterministic replays take a median 2.474 seconds when generated and 0.091 seconds on a cache hit, a 28.56× ratio that avoids

8,985 prompt and 384 completion tokens in total. It applies only to identical eligible requests. In a three-run semantic memory probe, one shared record is retrieved every time and the narrow rubric moves from 4/8 to 5/8. The extra record grows the prompt from 4,243 to 4,657 tokens and adds 0.239 seconds to median wall time. Memory is thus evidence for optional cross-thread context here, not for lower latency or token use. The sanitized rows are in `results/team_repository_reuse_qwen25_7b_20260809.json`.

6.4 Five-Developer Memory Systems Check

A deterministic harness creates five employee namespaces over two workspaces. It writes one reviewed shared checkout procedure, one private unfinished experiment, and one unrelated billing procedure. The five scenarios test shared retrieval for an adjacent issue, private isolation, workspace isolation, revision/dependency invalidation, and exact-request reuse for all five employees. All five checks pass. The exact-reuse fixture records five hits, 12,000 avoided prompt tokens, 600 avoided completion tokens, and USD 0.06 of avoided cost from supplied accounting values.

This harness executes no language model. Its token and cost totals verify key partitioning and metrics accumulation, not latency, throughput, answer quality, or likely production hit rate. The reproducible implementation is `scripts/benchmark_team_memory.py`; its assertions run in the package test suite.

6.5 Fresh-Prompt Block-Diffusion Results

The fresh-prompt suite contains one mathematical derivation, one bounded worker queue implementation, and one incident-response plan. Each target is loaded once per prompt family. Native and DFlash paths use the same target weights, chat template, greedy decoding, and 512-token cap. Three measured repetitions follow warm-up; medians below are taken over prompt-family medians.

Target	Verifier	Native	DFlash	Speedup	Acceptance
Qwen3.5 4B BF16	adaptive	28.25	46.81	1.64×	71.5%
Qwen3.5 9B 4-bit	adaptive	54.63	68.32	1.32×	67.8%
Qwen3.5 9B 4-bit	fixed 16	51.43	44.14	0.84×	72.9%

Table 3: Fresh-prompt decode throughput in tokens/s on the M5 Pro. The BF16 row isolates the decoding method. The 4-bit rows test a practical target and show why block length must adapt.

The 4B per-prompt medians are 2.54× for reasoning, 1.31× for code, and 1.64× for the operations plan. Its three accelerated continuations match native greedy output for all 128 validation tokens. The 9B adaptive medians are 1.73×, 1.22×, and 1.32×; fixed verification regresses code and operations to 0.84× and 0.76×. The adaptive 9B validation matches two of three suite prefixes. A separate two-sentence prompt diverges at token 21. These mismatches are retained as negative correctness evidence, so the 9B path is not promoted as equivalent.

The 4B BF16 row does not imply that it outruns every quantized implementation in absolute tokens/s. It answers a same-weight question: how much target decode work the block drafter removes. The 9B 4-bit row answers a deployment question and shows a smaller gain. Neither table entry includes model loading, and a short answer can remain dominated by prefill and first-token latency.

6.6 Native-Baseline Results

Profile/path	Match	Median	Base tok/s	MB tok/s	Accepted
Default/code	100%	1.96×	87.46	167.28	51
Default/RAG	100%	1.04×	89.98	91.61	0
Default/open	100%	1.00×	99.95	98.27	0
Re-entry/code	100%	1.62×	72.54	121.38	51
Re-entry/RAG	100%	1.58×	88.92	140.09	30
Re-entry/open	100%	1.08×	94.25	101.57	0

Table 4: Five fresh-nonce repeats per profile and fixture. Speedup is paired wall time; throughput and accepted-token columns are per-column medians. "MB" denotes MachBoost.

The default code fixture begins at a boundary that occurs in the supplied source. It accepts a median 51 of 64 output tokens and reduces logical target forwards by 76.6%. Its paired speedups are 2.44, 1.15, 2.08, 1.96, and 1.91×. The 1.96× median is close to the requested 2× threshold but does not clear it, and the range shows how noisy short local measurements remain.

The default retrieval answer has no candidate at its generation boundary and takes native fallback. With one-token re-entry and 1-gram lookup, the first target token anchors a context span; the verifier then accepts a median 30 tokens and reaches 1.58×. Open-ended generation accepts no draft tokens. Its apparent 1.08× ratio is timing noise rather than an algorithmic gain.

All 30 reported rows match their paired native token sequences. A separate 15-row exploratory run with a three-token re-entry probe had one RAG mismatch after an accepted span. This is why re-entry remains opt-in. For a user-facing system, the useful statistics are coverage, conditional speedup, and equality rate rather than one aggregate median.

6.7 Llama 3.2 Generalization Audit

A later audit tests seven workflow shapes on Llama 3.2 3B, with three alternating-order pairs per fixture and 64 greedy output tokens. Table 5 summarizes `results/llama32_3b_mlx_context_benchmark_20260716.json`.

Fixture	Rows	Exact pairs	Median speedup	Accepted tokens
Code completion	3	3/3	1.325×	47
Policy quotation	3	3/3	1.234×	34
Structured JSON	3	2/3	1.352×	34
Four no-draft fixtures	12	12/12	about 1.00×	0
Overall	21	20/21	1.008×	0

Table 5: Llama 3.2 3B context-drafting audit. Accepted tokens are per-fixture medians; the overall median is zero because most fixtures do not enter block verification.

The single mismatch occurs in a structured-JSON row after 34 accepted draft tokens. The artifact retains an identical decoded preview for the compared outputs but not the differing tail token, so it does not establish the cause. A strict control recomputes the full accepted prefix and records 9/9 exact pairs, but its 0.207× median makes it about 4.8× slower than native generation. This control is useful for diagnosis, not deployment. The audit narrows the earlier Qwen result: context drafting can help at eligible boundaries, while broad workload speed and exactness remain unresolved.

6.8 Resident Serving Results

The resident benchmark preloads the same 3B model, then sends five forced 64-token completions and five short streaming chats. It measures wall time at the server or Python streaming client, including prompt processing and detokenization. Table 6 summarizes the artifact `results/resident_qwen25_3b_20260713.json`.

Workload	Rows	Median TTFT	Median total	End-to-end tok/s
Forced 64-token completion	5	–	0.657 s	97.47
Short streaming chat	5	0.298 s	0.358 s	–

Table 6: Resident-server latency after preloading the model. Chat outputs contain nine tokens in each recorded row; TTFT is measured when the client receives the first non-empty text segment.

The first forced completion takes 0.973 seconds because the Metal execution path still needs initialization after weight loading; the following four take 0.650–0.663 seconds. These rows accept no context draft tokens. The measurement therefore demonstrates load amortization, corrected streaming, and native fallback rather than a speculative-decoding gain. An early, non-interleaved Ollama probe is retained in the repository as a historical diagnostic but is superseded by the comparison below.

A July 17 serving benchmark uses Llama 3.2 3B, two warm-up requests, seven measured requests, alternating runtime order, and a unique prompt nonce. It compares resident MachBoost MLX with Ollama 0.31.2. No context is supplied, so MachBoost follows native MLX generation. Table 7 summarizes `results/chat_latency_llama32_3b_20260717.json`.

Runtime	Rows	Median wall	Median TTFT	Decode tok/s
MachBoost MLX	7	0.679 s	0.247 s	144.00
Ollama	7	0.803 s	0.198 s	96.65

Table 7: Warm resident serving comparison. MachBoost and Ollama use different quantized model files and prompt templates; their generated outputs are not expected to be identical.

MachBoost is 1.18× faster by median wall time and 1.49× faster by reported decode throughput in this run. Ollama reaches first text about 1.25× sooner. These numbers compare two serving stacks, not the context-drafting algorithm or exact weights, and one machine-model pair is not enough for a general runtime ranking.

6.9 Repeated-Image Results

The visual benchmark uses a generated 1024 by 768 image containing a project name, status, budget, owner, and labeled colored shape. Four short extraction questions are each repeated three times. Every pair sends one request with both visual caches disabled and one request with both enabled; pair order alternates by repetition. Both paths use the same loaded model, greedy decoding, a 16-token limit, and the same image. Before recording, the harness performs one uncached warm-up and one cached prime. Table 8 summarizes `results/vision_cache_qwen25_3b_20260714.json`.

Metric	Uncached	Accelerated	Ratio
Median wall time	2.818 s	0.152 s	18.58×
Median paired wall time	–	–	18.33×
Median time to first text	2.807 s	0.144 s	19.45×
Effective prompt throughput	379.43 tok/s	12928.44 tok/s	34.07×

Table 8: Twelve uncached and 12 accelerated requests over one unchanged image. Effective prompt throughput divides the full logical prompt length by measured prefill time even when cached tokens are not recomputed.

The median of per-pair wall-time ratios is 18.33×; individual ratios are retained in the artifact. All 12 accelerated outputs exactly equal their paired uncached outputs, and both modes answer every fixture question correctly. Projected features hit in all recorded accelerated rows. Eleven rows reuse a median 1,018-token visual prefix and range from 13.32× to 21.36×. The feature-only control reaches 1.33×.

The distinction between those rows explains most of the result. Skipping the vision tower alone removes useful work, but the language model still processes the visual token span. Reusing its prompt state reduces the second stage to the changed question suffix. Generated answers are only two to five tokens long, so time to first text dominates total latency. Generation throughput after the first token is not improved by this mechanism.

The broader matrix uses the same image, questions, generation settings, warm-up policy, and alternating pair order. Table 9 summarizes `results/vision_cache_qwen_matrix_20260714.json`; each row contains 12 pairs.

Model	Total	Base	Cached	Paired	TTFT	Exact
Qwen3-VL 2B	2B	1.524 s	0.132 s	11.41×	12.23×	100%
Qwen3-VL 4B	4B	2.743 s	0.214 s	12.73×	13.32×	100%
Qwen3-VL 8B	9B	5.152 s	0.307 s	16.69×	17.30×	100%
Qwen3.5 0.8B	0.9B	0.656 s	0.125 s	5.14×	5.48×	75%
Qwen3.5 4B	5B	3.199 s	0.203 s	14.29×	16.43×	100%
Qwen3.5 9B	10B	5.572 s	0.311 s	17.44×	18.82×	100%

Table 9: Cross-model repeated-image results. “Paired” is the median of per-pair wall-time ratios. “Total” follows the official multimodal model listings rather than the variant label or quantized file size. Every baseline and cached row returns the expected fixture answer.

The median of the six model-level paired medians is 13.51×. All 144 measured responses return the expected answer. Literal equality holds in 69 of 72 pairs (95.83%); the three differences come from Qwen3.5 0.8B placing or omitting a semicolon around the same `BLUE SQUARE` answer inside a JSON fence. Qwen3-VL records no projected-feature hits, by design, and its three genuine no-prefix controls have a 0.99× median. Its remaining 33 cached rows reuse a 776-token prefix. All 36 Qwen3.5 cached rows restore a 776-token whole-state checkpoint.

The official Qwen3.6 collection lists a 27B variant with 28B total parameters and a 35B-A3B variant with 36B total parameters [18]. Neither meets the requested small-model constraint, even though the latter activates 3B parameters per token and quantized files can occupy less than 10 GB. Qwen3-VL 8B is listed as 9B total, while Qwen3.5 9B is listed as 10B total; the table keeps these boundary cases explicit.

6.10 Unique-Image Post-Fusion Pilot

The first-view harness draws ten unique image/question pairs from the public TextVQA dataset. Each pair compares the native full-token path with adaptive post-fusion compression at a requested 35% visual-token ratio. Both visual and prompt caches are disabled. Pair order alternates, and one held-out unique image warms each mode before recording. Generation is greedy with a 16-token limit. The artifact `results/cold_vision_qwen3vl_8b_postfusion_20260715.json` stores source identifiers, accepted dataset answers, raw model outputs, timing, cache counters, and retained-token telemetry.

Metric	Native	Post-fusion	Ratio/agreement
Median wall time	4.078 s	2.368 s	1.72×
Aggregate wall time	43.771 s	26.170 s	1.67×
Median paired wall time	–	–	1.70×
Median time to first text	4.029 s	2.351 s	1.71×
Accepted-answer match	80%	80%	unchanged
Normalized output equality	–	–	70%

Table 10: Ten unique TextVQA pairs on Qwen3-VL 8B. The accelerated path retains a median 35.12% of visual hidden states after layer 3. Ratios include vision encoding, language prefill, and short-answer decode, but exclude model load.

The median paired result is 1.70× and the ratio of total recorded times is 1.67×. Time to first text accounts for nearly all of the reduction, as expected for answers capped at 16 tokens. No row reports a visual-cache hit or reused prompt prefix. Native and compressed paths each match an accepted TextVQA answer on eight rows, but they are not token-equivalent: normalized outputs match on seven rows and literal outputs on five. Equal aggregate task score on this small sample does not show that errors are identical or that quality is preserved on the dataset as a whole.

6.11 Expanded Harness and Incomplete Runs

The version 0.5 harness adds ChartQA, DocVQA, MMMU, and TextVQA loaders, one shared native row per sample, deterministic random controls, layer and bucket sweeps, rotating method order, and paired bootstrap intervals. It writes an atomic checkpoint after each completed sample. We do not report a multi-dataset result from this harness because the run did not complete.

The failure occurred first on a fresh native Qwen3-VL 8B request for a 2257 by 1764 DocVQA page. Native prefill reached 3,945 tokens, but generation did not return before a 120-second request timeout with post-fusion compression disabled. After the stuck process was terminated, later model initialization attempts failed with a Metal command-buffer error. These sessions cannot distinguish algorithm latency from runtime or driver failure and are excluded from the evidence tables. The event also shows why per-dataset isolation, fail-fast timeouts, and checkpoints are necessary for local accelerator evaluation.

The temporal selector has a narrower integration check. A generated nine-second clip contains three static color scenes. Sampling at two frames per second produces 18 extracted frames; a 12-frame uniform budget and RGB change threshold retain four frames that cover the beginning, both color transitions, and the end. The 66.7% reduction relative to the uniform budget verifies selector behavior only. No VLM latency or task-quality result is inferred from it.

6.12 Why Earlier Ratios Were Rejected

Earlier repository artifacts reported 3–8× on MLX. Their baseline disabled the prompt cache or performed a synchronized target call for each token. That path ran near 10 tokens/s and was not representative of native `mlx-lm`, which uses cached asynchronous generation. Those artifacts remain available as implementation diagnostics, but they cannot support a production speed claim. Replacing that baseline was also what exposed a regression in MachBoost's serial tail; switching the tail to native generation restored parity for non-overlap prompts.

A one-repeat Hugging Face artifact likewise found MachBoost context lookup competitive with Transformers prompt lookup, but it predates the current paired harness. It is useful for designing a larger comparison, not for the headline result.

6.13 Exploratory Alternatives

We tested several ways to extend acceleration beyond literal context overlap. Table 11 summarizes the local probes. They were not all run under one benchmark protocol, so the numbers are diagnostic rather than comparative.

Path	Local result	Main constraint
Qwen2.5 0.5B draft for 3B target	0.38–0.78×	Acceptance of roughly 36–58% did not repay serial draft generation.
Ollama embedded MTP	12.7–24.3 tok/s versus roughly 40 tok/s baseline	The tested draft depths added overhead on this model and machine.
ANE 0.5B draft	51.1 decode tok/s	A one-token ANE draft cannot feed a 3B MLX target near 102 tok/s economically; multi-token heads are needed.
Lossless Q4 compression	1.08–1.15× ideal entropy bound	The quantized weights are already close to high entropy; decoder cost would reduce the gain further.

Table 11: Exploratory paths that did not beat native generation locally. Each result is specific to the tested model, implementation, and M1 Max. DFlash moved to Table 3 after testing its native M5 runtime.

The failures share a simple accounting problem. A draft only helps when the time removed from target decoding exceeds draft generation, verification, synchronization, and cache repair. A smaller model is not automatically a cheap enough draft. Moving it to another processor is not sufficient either if it predicts one token at a time. The strongest published results solve this with trained multi-token prediction, high acceptance, or concurrent heterogeneous execution rather than with process priority.

7 Limitations

The text-drafting evidence begins with 30 controlled Qwen rows and adds a 21-pair Llama 3.2 audit on the same machine. The favorable Qwen code fixture substantially contains the desired continuation; it demonstrates the mechanism but does not estimate real-world eligibility. In the broader Llama suite, most fixtures accept no tokens, the aggregate median is 1.008×, and one cache-enabled JSON pair is not token-identical. The strict verifier restores equality in its smaller control but is about 4.8× slower than native generation.

The DFlash matrix has only three prompt families, one machine, two target sizes, one draft family, and greedy decoding. The 512-token cap emphasizes decode and overstates the benefit for short answers. The 4B validation covers 384 generated tokens in total, not a task-quality benchmark. The 9B 4-bit divergence shows that target verification does not eliminate implementation-level non-invariance between batched and scalar MLX paths. This is consistent with an open `mlx-lm` report of greedy speculative divergence, but the local evidence does not identify the responsible kernel or prove semantic equivalence. Sampling, tool calls, long context, concurrent batching, energy, and model families outside the published DFlash registry remain untested.

The repository-prefix evidence now covers one 183-file controlled snapshot and one 8,754-file private production snapshot, six new-question pairs in the private audit, two Qwen2.5 model sizes overall, greedy generation, and one M5 Pro. Byte equality shows that cache reuse did not change these continuations; it does not prove that retrieval found all required code or that an answer would produce a correct patch. The private audit's small rubrics are especially weak: both paths score equally, but only four of eight and two of seven concepts are present. A request outside the workspace and the first request for a content revision receive no shared-prefix benefit. The 8,000-character evidence cap may omit relevant code in a broad or cross-cutting question. Longer sessions, multiple concurrent replicas, additional 3B–32B architectures, energy, and peak cache memory remain untested. A Qwen3.5 9B probe could not safely trim its hybrid recurrent cache and produced no valid speedup.

The five-developer memory check uses synthetic records and deterministic accounting. The private-repository probe adds three model-backed retrievals, but one narrow rubric improvement is not an estimate of general usefulness. In that probe memory increases tokens and latency. Automated summaries are redacted and bounded but are not a data-loss-prevention system. Shared-memory publication requires administrator action; the present evaluation does not study malicious content, poisoned memories, or conflicting decisions across branches.

The latest resident comparison alternates seven measured Llama requests between MachBoost and Ollama, but the stacks use different model files and templates. It measures serving suitability rather than an algorithmic speedup or quality comparison. The earlier Qwen resident artifact adds ten requests but does not broaden hardware coverage.

The repeated-image evidence spans six model sizes but remains one synthetic image, four extraction questions, three repeats, short answers, two closely related Qwen families, and one Apple Silicon machine. It excludes model load and starts after an explicit prime. The result says nothing about changed images, long-form visual reasoning, memory pressure under many cached images, or concurrent sessions. Qwen3-VL's no-prefix cache controls show no gain from cache machinery alone. Since decode work is unchanged, the ratio should shrink as answer length grows.

The first-view evidence is smaller: ten OCR-heavy TextVQA rows, one quantized Qwen3-VL 8B model, one retention ratio, one layer boundary, and one machine. It has neither exact output preservation nor enough samples for a quality-equivalence or statistical significance claim. Version 0.5 implements random, shape, depth, and automatic-policy ablations, but the multi-dataset run failed in the native backend before producing a complete artifact. We therefore still lack a direct measured comparison with random pruning, attention ranking, AdaptInfer, ZOO-Prune, or an optimized CUDA implementation. The 35% policy remains experimental rather than a generally selected operating point.

The video adapter selects image frames; it is not a video-language architecture. Its RGB difference can retain scene cuts but miss short actions, camera motion, semantically important small regions, or repeated scenes with different relevance to the question. It has no real-video quality benchmark and no evidence against PruneVid or EchoPrune. Audio has no MachBoost adapter.

Only greedy decoding is evaluated. Even then, cold and warm kernels can use different reduction shapes; the Qwen3.5 0.8B punctuation drift and the Llama structured-JSON tail mismatch are concrete examples of literal non-invariance. The latter artifact does not retain enough information to establish whether cache trajectory, a near-tied logit, or another implementation detail caused the difference. This does not establish equivalence for sampling or beam search. Ollama is also not accelerated through its public HTTP API.

Finally, this work introduces neither prompt lookup, prompt caching, nor visual-token pruning. The contribution is a set of local integrations, prompt layouts, safety boundaries, and paired artifacts: context drafting, repository retrieval with exact-prefix reuse, architecture-aware cache handling, and post-deepstack Qwen3-VL compression. A stronger research claim needs broader comparisons with LLMA, Prompt Lookup Decoding, PLD+, MLX-VLM's current server, production prefix-cache systems, and established VLM pruning baselines.

8 Next Experiments

There are two plausible tracks. The first keeps models unchanged. Before expanding re-entry, it should identify and eliminate the cache-trajectory mismatch seen in the Llama audit, then test an online eligibility predictor and native verifier hooks for llama.cpp/Ollama. The resident server makes this testable under long sessions; evaluation should report request coverage, cold and warm time to first token, decode throughput, peak memory, concurrency, energy, and exact outputs against each backend's own optimized generator.

Repository workspaces need a larger retrieval-quality evaluation independent of the latency harness. The private audit establishes realistic indexing and six new-question pairs, but the next run should use public issue-resolution and repository question-answering datasets so prompts, retrieved evidence, patches, and grader outputs can be inspected. It should compare full-context, conventional RAG, and prefix-oriented layouts; score citation recall, tests, and answer correctness; and sweep map size, evidence cap, answer length, repository revision, model size, and concurrent clients. Block-level KV reuse or continuous batching could extend the gain beyond a single exact prefix, but neither is implemented here.

The second track accepts model-specific preparation. The immediate decode study should sweep a public workload mixture, estimate expected utility from online acceptance and cycle cost, and select native, MTP, or block-diffusion decoding per request. AngelSpec reports that conversational MTP and code/mathematics diffusion drafters occupy different operating regions; the Apple Silicon question is whether that routing survives unified-memory contention. A multi-token drafter could also run on the Apple Neural Engine while target verification runs on the GPU, following the heterogeneous lesson of Mirror-SD. Quantized draft weights and windowed draft KV state may reduce memory traffic in long contexts. Each path needs an output gate, a native no-regression fallback, and concurrent-serving measurements.

The immediate visual work is external validity. After resolving or isolating the observed native high-resolution MLX failure, the checkpointed harness should run ChartQA, DocVQA, MMMU, TextVQA, spatial reasoning, and hallucination-sensitive datasets independently. It should add non-Qwen VLMs, sweep layer, retention ratio, image size, and answer length, and report confidence intervals, memory, energy, and task metrics. The implemented random control is the first comparison; attention ranking, AdaptInfer, and ZOO-Prune remain necessary to determine whether query-weighted merging contributes beyond token count alone. Automatic policies should remain disabled unless a model/task calibration set clears both latency and quality gates. Video evaluation should compare uniform sampling, RGB change selection, query-aware selection, PruneVid,

and EchoPrune on temporal question-answering benchmarks.

9 Conclusion

MachBoost now has five measured conditional paths, one unvalidated temporal adapter, and a namespaced team-memory serving path. In the private-repository audit, six independent-thread new-question pairs reuse 75–77% of prompt tokens. Prefill is 3.75–3.84× faster, while unchanged decode limits end-to-end speedup to 1.31–1.43×. All paired outputs are byte-identical under greedy generation, but small rubric scores show that equality is not the same as answer quality. Exact replay reaches 28.56× only for identical deterministic requests. Semantic memory retrieves prior team context but costs 414 extra prompt tokens and 0.239 seconds in its three-run probe. For fresh output, adaptive DFlash raises Qwen3.5 4B BF16 median decode throughput by 1.64×, with prompt-family medians from 1.31× to 2.54× and three of three 128-token validation prefixes exact. The practical 9B 4-bit control reaches 1.32× but does not always reproduce native MLX tokens; fixed verification regresses overall. Context drafting remains less settled: a favorable Qwen code fixture reaches 1.96×, while a broader Llama 3.2 3B suite is exact in 20 of 21 pairs and has a 1.008× aggregate median. Across six visual models, short repeated questions over one unchanged image produce model-level paired medians from 5.14× to 17.44×. For a new image, post-fusion Qwen3-VL compression reaches 1.67× aggregate speedup on a ten-row TextVQA pilot while both modes score 8/10; this path is approximate and normalized outputs agree on 70%. The evidence supports a local serving layer that selects among narrow mechanisms and refuses failed equivalence gates. It does not support a universal faster-inference claim.

References

[1] Apple Machine Learning Research. Mlx: An array framework for apple silicon. https://github.com/ml-explore/mlx, 2023. Software.

[2] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025.

[3] Nikhil Bhendawade, Kumari Nishu, Arnav Kundu, Chris Bartels, Minsik Cho, and Irina Belousova. Mirror speculative decoding: Breaking the serial barrier in llm inference. Apple Machine Learning Research, 2025.

[4] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

[5] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[6] Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026.

[7] Param Chordiya. Lossless but not free: An empirical anatomy of speculative decoding on consumer hardware. *arXiv preprint arXiv:2607.17283*, 2026.

[8] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2023.

[9] Xiaohu Huang, Hao Zhou, and Kai Han. PruneVid: Visual token pruning for efficient video large language models. *arXiv preprint arXiv:2412.16117*, 2024.

[10] Hugging Face. Assisted decoding. https://huggingface.co/docs/transformers/en/assisted_decoding, 2026. Accessed July 6, 2026.

[11] Youngeun Kim, Youjia Zhang, Huiling Liu, Aecheon Jung, Sunwoo Lee, and Sungeun Hong. ZOO-Prune: Training-free token pruning via zeroth-order gradient estimation in vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 39572–39582, 2026.

[12] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR, 2023.

[13] Jiameng Li, Minye Wu, Jiezhang Cao, Aleksei Tiulpin, and Matthew B. Blaschko. EchoPrune: Interpreting redundancy as temporal echoes for efficient VideoLLMs. *arXiv preprint arXiv:2605.10050*, 2026.

[14] Hong Liu, Rui Cen, Junhan Shi, Guangshuo Qin, Jiebin Zhang, Tianyu Liu, Runzhi Fan, Guoliang Zhao, Ruobing Xie, Kai Zhang, Song Liu, Guanghua Yu, and Jianchen Zhu. AngelSpec: Towards real-world high performance inference with speculative decoding. *arXiv preprint arXiv:2607.25852*, 2026.

[15] MLX-VLM contributors. MLX-VLM: Vision language models on apple silicon. https://github.com/Blaizzy/mlx-vlm, 2026. Software, accessed July 14, 2026.

[16] Qwen Team. Qwen3-VL model collection. https://huggingface.co/collections/Qwen/qwen3-vl, 2026. Accessed July 15, 2026.

[17] Qwen Team. Qwen3.5 model collection. https://huggingface.co/collections/Qwen/qwen35, 2026. Accessed July 15, 2026.

[18] Qwen Team. Qwen3.6 model collection. https://huggingface.co/collections/Qwen/qwen36, 2026. Accessed July 15, 2026.

[19] Apoorv Saxena. Prompt lookup decoding. https://github.com/apoorvumang/prompt-lookup-decoding, 2023. GitHub repository.

[20] Shwetha Somasundaram, Anirudh Phukan, and Apoorv Saxena. Pld+: Accelerating llm inference by leveraging language model artifacts. *arXiv preprint arXiv:2412.01447*, 2024.

[21] Rishabh Tiwari, Haocheng Xi, Aditya Tomar, Coleman Hooper, Sehoon Kim, Maxwell Horton, Mahyar Najibi, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. Quantspec: Self-speculative decoding with hierarchical quantized kv cache. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

[22] Yahong Wang, Juncheng Wu, Zhangkai Ni, Longzhen Yang, Yihang Liu, Chengmei Yang, Ying Wen, Lianghua He, Xianfeng Tang, Hui Liu, and Yuyin Zhou. When token pruning is worse than random: Understanding visual token information in VLLMs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 31910–31919, 2026.

[23] Fabian Waschkowski, Prabod Rathnayaka, and Lukas Wesemann. BaseRT: Advancing best-in-class LLM inference with apple M5 neural accelerators. *arXiv preprint arXiv:2607.19438*, 2026.

[24] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics, 2020.

[25] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[26] Nan Yang, Tao Ge, Liang Wang, Binxing Jiao, Daxin Jiang, Linjun Yang, Rangan Majumder, and Furu Wei. Inference with reference: Lossless acceleration of large language models. *arXiv preprint arXiv:2304.04487*, 2023.

[27] Aonan Zhang, Ray Zhang, Yunfei Cheng, Chong Wang, and Yi Wang. Recurrent drafter for fast speculative decoding in large language models. *arXiv preprint arXiv:2403.09919*, 2024.

[28] Weichen Zhang, Zhui Zhu, Ningbo Li, Kebin Liu, and Yunhao Liu. AdaptInfer: Adaptive token pruning for vision-language model inference with dynamical text guidance. *arXiv preprint arXiv:2508.06084*, 2025.

[29] Xin Zhou, Dingkang Liang, Kaijin Chen, Tianrui Feng, Xiwu Chen, Hongkai Lin, Yikang Ding, Feiyang Tan, Hengshuang Zhao, and Xiang Bai. Less is enough: Training-free video diffusion acceleration via runtime-adaptive caching. *arXiv preprint arXiv:2507.02860*, 2025.